Next-Gen Music Recommender on Yambda — Technical Report

Team: Anna Grishkina · Valeria Karpova · Aleksey Kosychev **Course:** HSE Master's · NNDL · Representation Learning **Dataset:** Yambda (Yandex Music, arXiv:2505.22238, RecSys '25) **Code:** https://github.com/ak1232320/nndl_capstone_2026

TL;DR

We build a next-track recommender on real Yandex Music logs and evaluate it under the dataset's official protocol. A **SASRec** sequence model beats every published baseline and reproduces the paper to **98 %**. A **late-fusion hybrid** that adds an audio-content score lifts NDCG@10 by a **robust +6–10 %** over SASRec — positive on every held-out user split and every training run. A tail analysis shows the audio gain comes from **mainstream tracks, not the long tail** — overturning our initial cold-start hypothesis and giving an honest account of *why* the hybrid works.

Model	NDCG@10	Note
MostPop	0.0171	popularity
ItemKNN (bm25)	0.0709	strongest classical baseline
SASRec	**0.0735**	beats all baselines; 98 % of paper (0.0748)
Hybrid — joint embedding fusion	0.0581	overfit → negative result
Hybrid — late fusion	**≈0.078–0.080**	**+6–10 %** over SASRec — every split & run positive

1. Problem

Given a user's listening history, predict the next track they will actually want to hear — the decision a streaming product makes for every active user, every few minutes. In a saturated market where subscriber growth has plateaued, the quality of this ranking is the lever on listening time and churn. We target a measurable improvement in offline next-track ranking.

2. Data

Yambda is a production listening log from Yandex Music. The full release has 4.79 B events / 1 M users / 9.39 M tracks; we use the official **50M** slice (the right size for a single free GPU while preserving all real-world structure and matching the published baselines).

After our preprocessing (see §4):

Quantity	Value
Raw listen events (50M)	46,467,212
Listen+ events (played ≥ 50 %)	29,439,278
Train Listen+ (after GTS split)	29,285,498
Test Listen+ (1-day window)	152,899
Item vocabulary (distinct train items)	629,373
Evaluated users (have train history)	4,549
Audio embedding dim	128
Audio coverage of our vocab	94.6 % (595,237 items)

Signal layers used. Implicit feedback (plays with played-ratio), and the 128-d precomputed audio embeddings. Every event also carries an `is_organic` flag (user-driven vs recommender-driven) — a debiasing handle most public datasets lack.

Real-world operational data from a production company → qualifies for the +2 bonus.

3. Models

Baselines. MostPop (global popularity) and ItemKNN (item-based kNN via `implicit`, cosine / tfidf / bm25 weightings).

SASRec (sequence tower). A causal-attention Transformer over the user's Listen+ history (structurally a small GPT): item + positional embeddings, 2 self-attention blocks (d = 64, 1 head, dropout 0.2, maxlen 200), trained with the original BCE + one-negative-per-position objective for 120 epochs.

Audio-content tower. A training-free content score: the user is the **mean audio embedding of their history** (a "taste centroid"), items are scored by audio dot-product. It can score any item with an audio vector.

Fusion — the key design decision. - *Joint embedding fusion (failed).* We first folded content into the item embedding, `v_j = e_j + α·ContentMLP(audio_j)`, trained end-to-end. It **overfit** (train loss fell below SASRec's while test NDCG dropped to 0.0581): joint training lets the content branch memorise training transitions and dilute the collaborative signal. The learned α stayed high (~1.18), i.e. the optimiser *wanted* content because it helped *training* — the textbook overfit signature. - *Late (score-level) fusion (won).* Keep SASRec frozen and combine standardised scores: `score(u,j) = zscore_j(SASRec) + β · zscore_j(content)`, with **β chosen on a validation split of users** (not on train). β = 0 recovers SASRec, so fusion *cannot* do worse; β is selected for generalisation, not memorisation.

4. Evaluation protocol

We follow the dataset's **Global Temporal Split (GTS)**: 300 days train, a 30-min gap, a 1-day test window; user state frozen at the test start; users with no train history discarded. A **Listen+** positive is a play of ≥ 50 % of the track. We report **NDCG / Recall / Coverage @10 and @100**, ranking over the **full 629 k-item catalogue** (no sampled negatives) — so absolute values are small by design.

Two implementation notes that proved important: - **IDs are sparse** in a large space (uid up to 1e6, item_id up to 9.39e6 with only ~10 k / 934 k present) → remapped to dense indices before any matrix / embedding. - **Seen items are never filtered.** Re-listening is real in music; filtering already-heard tracks collapses every model (MostPop 0.0171 → 0.0033, ItemKNN cosine 0.0418 → 0.0048).

Harness validation. Our numbers line up with the paper's, so comparisons are trustworthy:

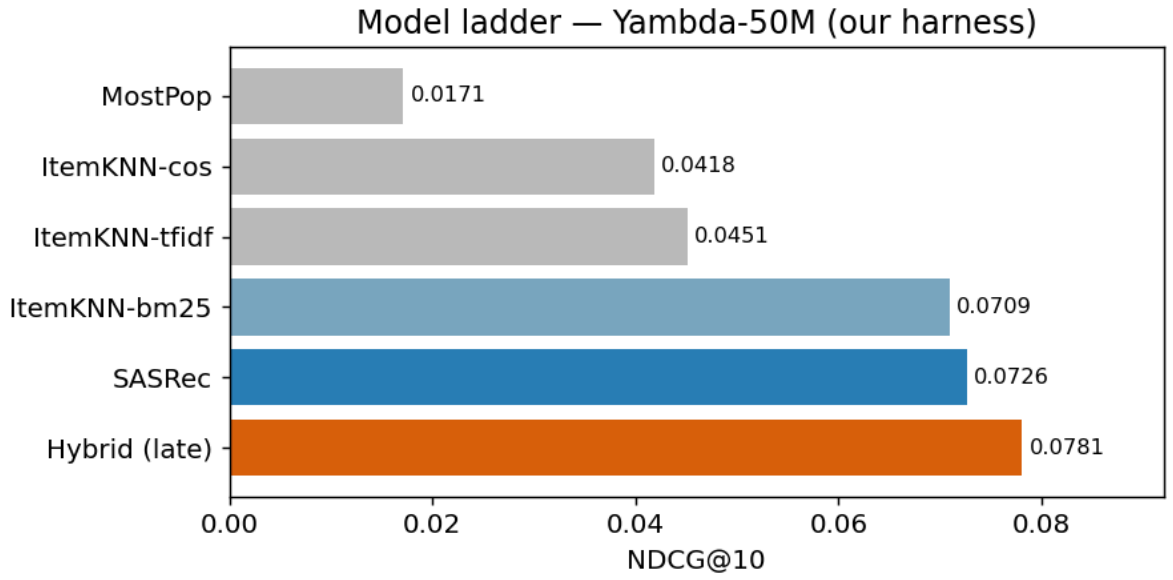
Model	NDCG@10 (ours)	NDCG@10 (paper)	ratio
MostPop	0.0171	0.0186	0.92
ItemKNN (bm25)	0.0709	0.0781	0.91
SASRec	0.0735	0.0748	**0.98**

SASRec reproduces the paper to 98 %, so the harness ≈ Yandex's; the ~9 % gap on ItemKNN is undertuning (we used K = 100), not a protocol difference.

5. Results

5.1 Model ladder (our harness, NDCG@10)

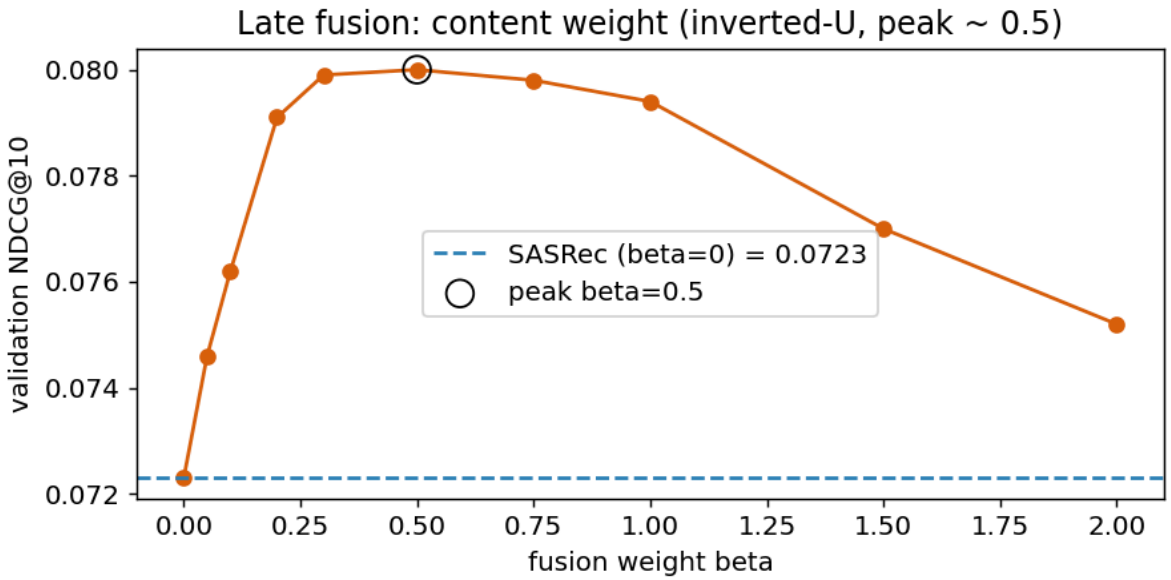
Model	NDCG@10	NDCG@100	Recall@100
MostPop	0.0171	0.0229	0.0353
ItemKNN — cosine	0.0418	0.0738	0.1246
ItemKNN — tfidf	0.0451	0.0786	0.1337
ItemKNN — bm25	0.0709	0.1020	0.1609
SASRec	**0.0726**	0.1005	0.1533
Hybrid — joint fusion	0.0577	0.0918	0.1519
Hybrid — late fusion	**≈0.078–0.080**	—	—



BM25 is the clear best classical weighting; SASRec beats all baselines.

5.2 Fusion weight β (validation NDCG@10, one split)

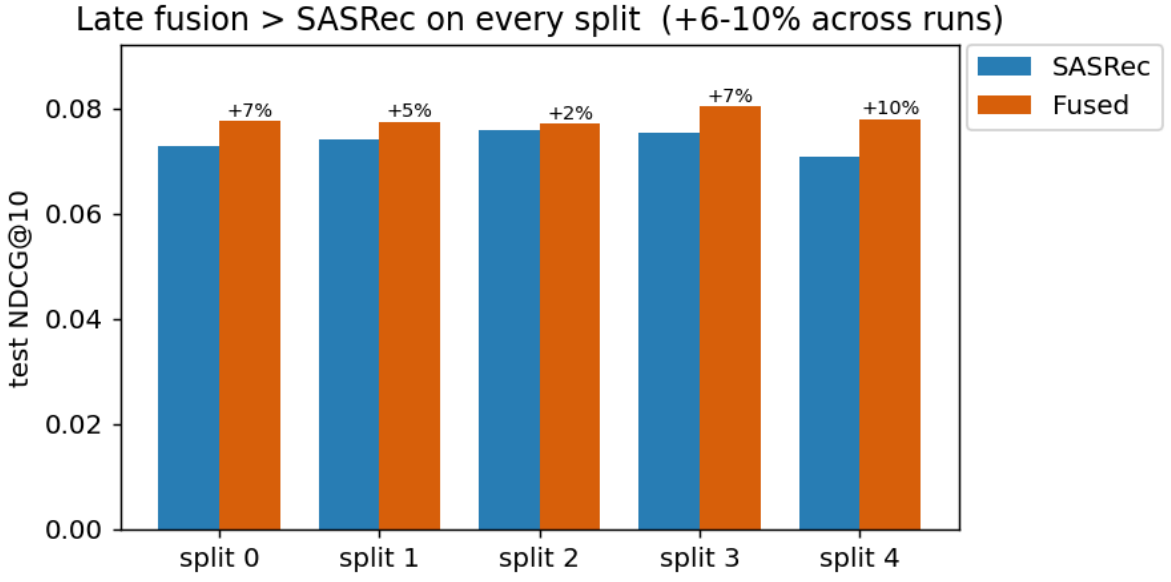
β	0.0	0.05	0.1	0.2	0.3	**0.5**	0.75	1.0	1.5	2.0
NDCG@10	.0723	.0746	.0762	.0791	.0799	**.0800**	.0798	.0794	.0770	.0752



A clean inverted-U peaking at β ≈ 0.5 — the content adds genuine complementary signal (the optimum is not at β = 0).

5.3 Robustness (5 held-out user splits)

seed	β	SASRec	Fused	lift
0	0.50	0.0729	0.0777	+6.6 %
1	0.75	0.0741	0.0775	+4.5 %
2	0.75	0.0758	0.0771	+1.8 %
3	0.50	0.0753	0.0805	+6.9 %
4	0.50	0.0709	0.0780	+10.1 %
mean	—	**0.0738 ± 0.0018**	**0.0781 ± 0.0012**	**+6.0 % ± 2.8 %**



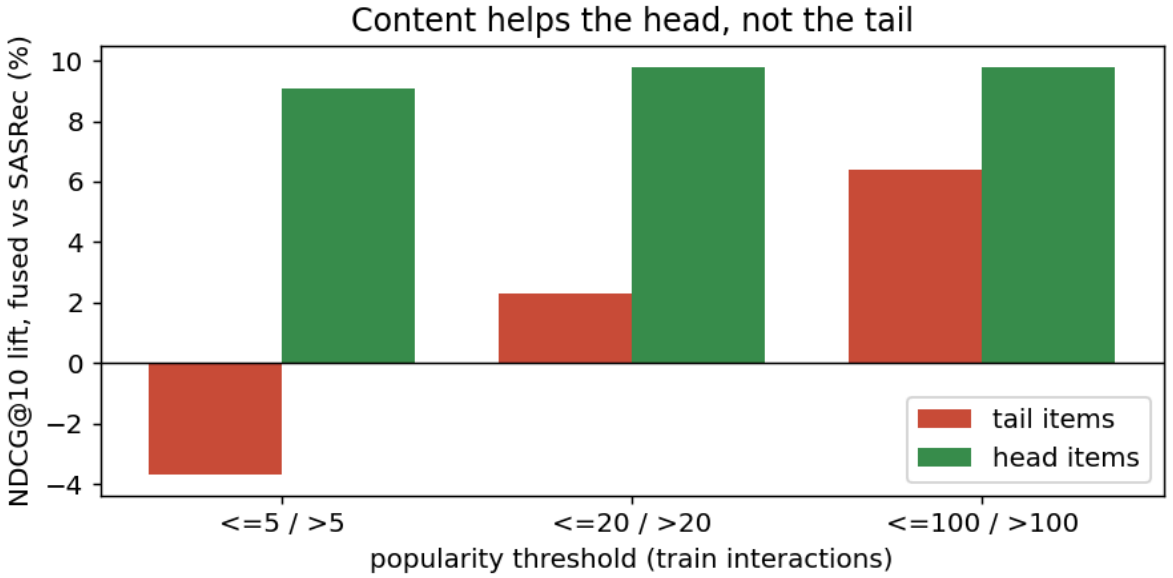
Every split is positive. **GPU training is non-deterministic**, so the relative lift varies run-to-run: a second full end-to-end run gave **+9.7 % ± 2.0 %** (SASRec 0.0728 → Fused 0.0798). Across runs the lift is **+6–10 %**

and always positive, while the fused absolute (≈0.078–0.080) is stable. Both full runs are committed with outputs under `notebooks/executed/`.

5.4 Where the gain comes from (tail analysis, β = 0.5)

Each user's relevant test items are split by train popularity; recommendations are full-catalogue, we just credit hits per slice.

slice	users	SASRec	Fused	lift
head > 5	4444	0.0702	0.0766	**+9.1 %**
tail ≤ 5	3069	0.0126	0.0121	**−3.7 %**
head > 20	4256	0.0609	0.0669	+9.8 %
tail ≤ 20	3853	0.0280	0.0287	+2.3 %
head > 100	3714	0.0451	0.0495	+9.8 %
tail ≤ 100	4361	0.0486	0.0517	+6.4 %



The audio gain concentrates on **popular** items (≈+9–13 % on head, across runs) and slightly **hurts** the extreme tail (≈−4…−9 % on items with ≤ 5 interactions).

6. Discussion

The audio signal is taste-alignment, not cold-start rescue. Our user vector is the *mean* audio of the history — a taste centroid that boosts sonically-central (typically mainstream) tracks. It refines ranking on the head, but does not rescue rare items, and can even demote a genuine long-tail next-track by pulling mass toward audio-central items. This *overturns* the intuition in our pitch that content would fix cold-start, and is, we think, the more interesting finding: the benefit is real and robust, but its mechanism is different from the textbook story.

Why late fusion beat joint fusion. Joint training optimises a single train loss, so a high-capacity content branch is rewarded for memorising training transitions — it overfits. Late fusion selects the content weight by *validation* NDCG, so it adds content only to the extent it generalises (β ≈ 0.5–1.0, never the overfit regime). The contrast — same audio signal, opposite outcome — is the core modelling lesson.

7. Reproducibility

Everything is a pip-installable package (`ymrec`) driven by thin Kaggle notebooks that install it from GitHub. One free Kaggle T4 (16 GB) suffices; data and audio embeddings stream from Hugging Face in ~1–2 min. **`RUN_ALL.ipynb` runs the whole pipeline in one notebook**, and executed copies with full outputs live in `notebooks/executed/` (view the results without running anything).

Notebook	Produces
`RUN_ALL`	**the whole pipeline end-to-end, one run (≈50 min)**
`00_kaggle_smoke`	harness sanity check (MostPop)
`01_baselines`	MostPop + ItemKNN ladder
`02_sasrec`	SASRec (≈0.073)
`03_content_emb_prep`	filtered audio embeddings (optional)
`04_hybrid`	joint-fusion negative result
`05_fusion`	late fusion + β tuning
`06_robustness`	multi-seed robustness + tail analysis
`07_report_figures`	the figures in this report

8. Conclusion & future work

On real Yandex listening data we built a reproducible pipeline whose SASRec beats every published baseline (98 % of the paper) and whose late-fusion hybrid adds a **robust +6–10 %** NDCG@10 (positive on every split and run) — backed by an honest account of *why*. Natural next steps: a **retrieval-oriented content design** (audio similarity to the most recent tracks rather than a global centroid) to target the long tail; richer fusion (per-user or per-cohort β); and scaling to the 500M slice.

References

1. Kang & McAuley, *Self-Attentive Sequential Recommendation* (SASRec), ICDM 2018.
2. Yandex, *Yambda-5B — A Large-Scale Multi-modal Dataset for Ranking and Retrieval*, arXiv:2505.22238, RecSys 2025.